

2.AI协作开发模型

核心定位

不是：

AI 写代码，人负责运行。

而是：

人负责理解、决策和验证；AI负责加速检索、实现和工程执行。

原因是现在软件开发的瓶颈正在从“输入代码”转向“判断代码是否正确、为什么这样设计、如何演进”。很多工程实践也强调，AI生成代码之后，人类的审查、验证和设计判断仍然是关键环节。[Addy Osmani](#)

一、项目推进模式：同步双轨 + 功能切片

以后默认采用：

确定一个最小功能目标



理解原理和系统位置



阅读官方资料/源码



明确调用链、状态和接口



AI辅助实现业务代码



编译、运行、调试



自己解释、修改、重建



进入下一个功能切片

而不是：

先花几周把所有理论学完



最后开始写代码

也不是：

AI一次生成完整系统



人慢慢补理解

后者容易形成“理解债务”：代码越来越多，但是开发者不知道为什么这样组织。[Addy Osmani](#)

二、AI负责什么

以后 Codex / 工具型 AI 更适合承担：

1. 信息获取

例如：

- 阅读 SDK；
 - 找 API；
 - 查官方示例；
 - 查头文件；
 - 查构建关系；
 - 分析调用链。
-

2. 工程实现

例如：

- 创建工程骨架；
 - 编写重复代码；
 - 补充 CMake；
 - 写日志；
 - 重构代码；
 - 修改已有模块。
-

3. 工程验证辅助

例如：

- 分析编译错误；
- 定位异常日志；
- 检查代码风险；

- 提供测试方案。

三、你必须掌握什么

以后评价“这个技术你是否真正掌握”，不看：

你是否能背下来所有 API。

而看：

1. 能解释

比如：

这个函数：

```
sle_xxx_init()
```

你需要知道：

- 它属于哪一层？
- 为什么需要调用？
- 谁调用它？
- 它改变了什么状态？

2. 能追踪

例如：

RSSI读取：

```
系统启动
↓
初始化SLE
↓
建立连接
↓
保存conn_id
↓
启动周期任务
↓
调用RSSI接口
↓
```

等待回调



输出结果

你能从头走完整链路。

3. 能预测

例如：

如果：

- 删除广播启动；
- 不注册连接回调；
- 修改采样周期；
- conn_id保存错误；

你知道可能出现什么现象。

4. 能调试

出现：

- 扫不到设备；
- 连不上；
- RSSI不返回；
- 数据异常；

你知道应该在哪一层排查。

5. 能重建

不是要求：

闭卷默写整个 SDK。

而是：

给你需求，你能设计结构、写伪代码，然后查官方 API 补齐细节。

这才符合真实工程。

四、代码哪些自己写，哪些交给 AI

以后默认：

内容	方式
系统架构	你决定
模块划分	你参与设计
状态机	你理解并确认
核心通信链路	你必须掌握
关键回调关系	你必须掌握
数据结构设计	你必须理解
重复代码	AI生成
模板代码	AI生成
API参数查询	AI辅助
构建配置	AI辅助
调试分析	人机协作

五、对于你当前星闪项目，具体执行方式

当前第一阶段不会改变：

切片1：最小 Server

目标：

你能理解并写出：



Codex：

- 查官方 Server;
- 建立你的业务 Server;
- 编译验证。

你:

- 理解调用链;
- 能解释;
- 能修改关键参数。

切片2：最小 Client

目标:

```
启动
↓
扫描
↓
发现目标
↓
建立连接
↓
保存conn_id
```

切片3：RSSI闭环

目标:

```
连接
↓
周期读取RSSI
↓
RSSI回调
↓
串口输出
```

暂时不加入:

- 四锚点;
- 滤波;
- 距离模型;
- AI定位模型。

六、以后遇到任何项目，我会优先问

不是：

“要不要直接写代码？”

而是：

1. 当前最小可验证功能是什么？
2. 这个功能位于系统哪一层？
3. 我们需要先理解什么调用链？
4. 哪些部分必须由你掌握？
5. 哪些部分可以交给 AI 加速？
6. 完成标准是什么？

这套方式其实很适合你目前阶段：你不是纯新手，也不是已经成熟工程师，而是在从“会调用工具”过渡到“能够独立设计和维护系统”。

后续无论是：

- 星闪；
- ESP32；
- 后端；
- Linux；
- 数据库项目；

我都会优先按这个协作模型推进。